DATABASE DESIGN DOCUMENTS

(DDD)

TOPIC: WAREHOUSE & SUPPLY CHAIN

I/ Use Data Definition Language (DDL) Scripts

```
CREATE DATABASE IF NOT EXISTS lab1;

USE lab1;


CREATE TABLE Warehouse (

    WarehouseID INT NOT NULL AUTO_INCREMENT COMMENT 'Unique identifier of a warehouse',

    WarehouseName VARCHAR(100) NOT NULL COMMENT 'Warehouse name',

    Address VARCHAR(200) NOT NULL COMMENT 'Physical address',

    City VARCHAR(100) NOT NULL COMMENT 'City where the warehouse is located',

    Country VARCHAR(100) NOT NULL COMMENT 'Country where the warehouse is located',

    CapacityM3 DECIMAL(12,2) NOT NULL CHECK (CapacityM3 >= 0) COMMENT 'Maximum storage capacity in cubic meters',


    CONSTRAINT pk_warehouse_id PRIMARY KEY (WarehouseID)

);

CREATE INDEX idx_warehouse_name ON Warehouse(WarehouseName) COMMENT 'Supports searching warehouses by name';


CREATE TABLE Product (

    ProductID   INT NOT NULL AUTO_INCREMENT COMMENT 'Unique identifier of a product',

    ProductName VARCHAR(150) NOT NULL COMMENT 'Product name',

    Description TEXT NULL COMMENT 'Additional product details',

    Unit        VARCHAR(20) NOT NULL COMMENT 'Base unit used to measure quantity',
```

```
    ProductType VARCHAR(50) COMMENT 'Product category; allowed values are not finalized in Phase 1',


    CONSTRAINT pk_product_id PRIMARY KEY (ProductID)

);

CREATE INDEX idx_product_name ON Product(ProductName) COMMENT 'Supports product-name lookups';


CREATE TABLE Supplier (

    SupplierID   INT NOT NULL AUTO_INCREMENT COMMENT 'Unique identifier of a supplier',

    SupplierName VARCHAR(150) NOT NULL COMMENT 'Supplier business name',

    ContactName  VARCHAR(100) COMMENT 'Main contact person',

    Phone        VARCHAR(30) COMMENT 'Supplier contact telephone number',

    Email        VARCHAR(150) COMMENT 'Supplier contact email; not unique unless the business requires it',

    Address      VARCHAR(200) COMMENT 'Supplier business address',


    CONSTRAINT pk_supplier_id PRIMARY KEY (SupplierID)

);

CREATE INDEX idx_supplier_name ON Supplier(SupplierName) COMMENT 'Supports supplier-name lookups';


CREATE TABLE Inventory (

    InventoryID INT NOT NULL AUTO_INCREMENT COMMENT 'Unique inventory record identifier',

    WarehouseID INT NOT NULL COMMENT 'Warehouse holding the product',

    ProductID   INT NOT NULL COMMENT 'Product held at the warehouse',

    Quantity    DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK (Quantity >= 0) COMMENT 'Current stock balance',

    ReorderLevel DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK (ReorderLevel >= 0) COMMENT 'Threshold below which a restock alert is opened',
```

```
    LastUpdated DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP COMMENT 'Last inventory record update time',


    CONSTRAINT pk_inventory_id PRIMARY KEY (InventoryID),

    CONSTRAINT uq_inventory_warehouseid_productid UNIQUE (WarehouseID,
ProductID),

    CONSTRAINT fk_inventory_warehouse_id FOREIGN KEY (WarehouseID)
REFERENCES Warehouse(WarehouseID),

    CONSTRAINT fk_inventory_product_id FOREIGN KEY (ProductID)
REFERENCES Product(ProductID)

);


CREATE TABLE SupplierProduct (

    SupplierID   INT NOT NULL COMMENT 'Supplier offering the product',

    ProductID    INT NOT NULL COMMENT 'Product offered by the supplier',

    UnitPrice    DECIMAL(12,2) NOT NULL CHECK (UnitPrice >= 0) COMMENT
'Current price offered by this supplier',

    LeadTimeDays INT NOT NULL DEFAULT 0 CHECK (LeadTimeDays >= 0)
COMMENT 'Expected supplier delivery lead time in days',


    CONSTRAINT pk_supplierproduct_supplierid_productid PRIMARY KEY
(SupplierID, ProductID),

    CONSTRAINT fk_supplierproduct_supplier_id FOREIGN KEY (SupplierID)
REFERENCES Supplier(SupplierID),

    CONSTRAINT fk_supplierproduct_product_id FOREIGN KEY (ProductID)
REFERENCES Product(ProductID)

);


CREATE TABLE SupplierOrder (

    OrderID             INT NOT NULL AUTO_INCREMENT COMMENT 'Unique
supplier order identifier',

    OrderDate           DATE NOT NULL COMMENT 'Date the order was created',
```

```
    ExpectedDeliveryDate DATE COMMENT 'Expected date at the destination warehouse',
    Status               VARCHAR(20) NOT NULL DEFAULT 'Draft' COMMENT 'Lifecycle status; Draft and Submitted are used by the report rules',
    SupplierID           INT NOT NULL COMMENT 'Supplier receiving the purchase order',
    WarehouseID          INT NOT NULL COMMENT 'Destination warehouse for the order',
    CONSTRAINT pk_order_id PRIMARY KEY (OrderID),
    CONSTRAINT fk_order_supllier_id FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID),
    CONSTRAINT fk_order_warehouse_id FOREIGN KEY (WarehouseID) REFERENCES Warehouse(WarehouseID)
);
CREATE INDEX idx_order_status_date ON SupplierOrder(Status, OrderDate) COMMENT 'Supports filtering orders by status and date';

CREATE TABLE SupplierOrderItem (
    OrderItemID INT NOT NULL AUTO_INCREMENT COMMENT 'Unique order line identifier',
    OrderID     INT NOT NULL COMMENT 'Supplier order containing this line',
    ProductID   INT NOT NULL COMMENT 'Product being ordered',
    Quantity    DECIMAL(12,2) NOT NULL CHECK (Quantity > 0) COMMENT 'Quantity requested on this order line',
    UnitPrice   DECIMAL(12,2) NOT NULL CHECK (UnitPrice >= 0) COMMENT 'Historical price agreed for this order line',

    CONSTRAINT pk_orderitem_id PRIMARY KEY (OrderItemID),
    CONSTRAINT uq_orderitem_orderid_productid UNIQUE (OrderID, ProductID),
    CONSTRAINT fk_orderitem_order_id FOREIGN KEY (OrderID) REFERENCES SupplierOrder(OrderID),
    CONSTRAINT fk_orderitem_product_id FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
```

```
);


CREATE TABLE Shipment (
    ShipmentID      INT NOT NULL AUTO_INCREMENT COMMENT 'Unique shipment identifier',
    OrderID         INT NOT NULL COMMENT 'Supplier order fulfilled by this shipment',
    TrackingNumber  VARCHAR(50) COMMENT 'External carrier tracking reference',
    ShipmentDate    DATE COMMENT 'Date the shipment is dispatched',
    EstimatedArrival DATETIME COMMENT 'Current estimated arrival date and time',
    ActualArrival   DATETIME COMMENT 'Confirmed arrival date and time',
    Status          VARCHAR(20) NOT NULL DEFAULT 'Draft' COMMENT 'Shipment lifecycle status',


CONSTRAINT pk_shipment_id PRIMARY KEY (ShipmentID),
    CONSTRAINT fk_shipment_order_id FOREIGN KEY (OrderID) REFERENCES SupplierOrder(OrderID)
);
CREATE INDEX idx_shipment_status ON Shipment(Status);


CREATE TABLE ShipmentItem (
    ShipmentItemID INT NOT NULL AUTO_INCREMENT COMMENT 'Unique shipment line identifier',
    ShipmentID     INT NOT NULL COMMENT 'Shipment containing this allocation',
    OrderItemID    INT NOT NULL COMMENT 'Supplier order line being delivered',
    Quantity       DECIMAL(12,2) NOT NULL CHECK (Quantity > 0) COMMENT 'Quantity allocated to this shipment',


    CONSTRAINT pk_shipmentitem_id PRIMARY KEY (ShipmentItemID),
    CONSTRAINT fk_shipmentitem_shipment_id FOREIGN KEY (ShipmentID) REFERENCES Shipment(ShipmentID),
```

```
    CONSTRAINT fk_shipmentitem_orderitem_id FOREIGN KEY (OrderItemID)
REFERENCES SupplierOrderItem(OrderItemID)

);


CREATE TABLE TrackingEvent (

TrackingEventID INT NOT NULL AUTO_INCREMENT COMMENT 'Unique identifier
for a tracking event, including events at the same time',

    ShipmentID      INT NOT NULL COMMENT 'Shipment associated with this event',

    EventTime       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
COMMENT 'Event time',

    Location        VARCHAR(150) COMMENT 'Location where the event was
recorded',

    Status          VARCHAR(30) NOT NULL COMMENT 'Status reported by the
tracking event',

    Description     TEXT COMMENT 'Additional tracking or handling details',


    CONSTRAINT pk_trackingevent_id PRIMARY KEY (TrackingEventID),

    CONSTRAINT fk_trackingevent_shipment_id FOREIGN KEY (ShipmentID)
REFERENCES Shipment(ShipmentID)

);

CREATE INDEX idx_tracking_shipment_time ON TrackingEvent(ShipmentID,
EventTime, TrackingEventID) COMMENT 'Supports chronological tracking history
per shipment';


CREATE TABLE ProductComponent (

    ParentProductID     INT NOT NULL COMMENT 'Product assembled from
components',

    ComponentProductID INT NOT NULL COMMENT 'Direct component used by the
parent product',

    QuantityRequired    DECIMAL(12,2) NOT NULL CHECK (QuantityRequired > 0)
COMMENT 'Number of component units required for one parent unit',
```

```
    CONSTRAINT pk_productcomponent_id PRIMARY KEY (ParentProductID,
ComponentProductID),

    CONSTRAINT ck_productcomponent CHECK (ParentProductID <>
ComponentProductID),

    CONSTRAINT fk_productcomponent_parent_id FOREIGN KEY
(ParentProductID) REFERENCES Product(ProductID),

    CONSTRAINT fk_productcomponent_component_id FOREIGN KEY
(ComponentProductID) REFERENCES Product(ProductID)

);


CREATE TABLE StockMovement (

    MovementID      INT NOT NULL AUTO_INCREMENT COMMENT 'Unique stock
transaction identifier',

    InventoryID     INT NOT NULL COMMENT 'Inventory balance affected by this
movement',

    ShipmentItemID INT UNIQUE COMMENT 'Required only for Receipt movements;
each shipment item can be received once',

    MovementTime    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
COMMENT 'Time the movement is recorded',

    MovementType    VARCHAR(20) NOT NULL COMMENT 'Opening, Receipt, Issue,
or Adjustment',

    QuantityDelta  DECIMAL(12,2) NOT NULL COMMENT 'Signed change in
inventory quantity',


    CONSTRAINT uq_stockmovementid_shipmentitemid UNIQUE(MovementID,
ShipmentItemID),

CONSTRAINT pk_stockmovement_id PRIMARY KEY(MovementID),

    CONSTRAINT fk_stockmovement_inventory_id FOREIGN KEY (InventoryID)
REFERENCES Inventory(InventoryID),

    CONSTRAINT fk_stockmovement_shipmentitem_id FOREIGN KEY
(ShipmentItemID) REFERENCES ShipmentItem(ShipmentItemID)

);
```

```
CREATE INDEX idx_movement_inventory_time ON StockMovement(InventoryID, MovementTime, MovementID) COMMENT 'Supports movement history lookup by inventory and time';


CREATE TABLE RestockAlert (

AlertID              INT NOT NULL AUTO_INCREMENT COMMENT 'Unique low-stock alert period identifier',

   InventoryID          INT NOT NULL COMMENT 'Inventory record that triggered the alert',

   AlertDate            DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT 'Time the low-stock condition was detected',

   QuantityAtAlert      DECIMAL(12,2) NOT NULL COMMENT 'Quantity snapshot when this alert opened',

   ReorderLevelAtAlert  DECIMAL(12,2) NOT NULL COMMENT 'Reorder level snapshot when this alert opened',

   ResolvedAt           DATETIME COMMENT 'Resolution time; NULL means alert is unresolved',

   Message              VARCHAR(200) COMMENT 'Human-readable alert explanation',


   CONSTRAINT pk_restockalert_id PRIMARY KEY(AlertID),

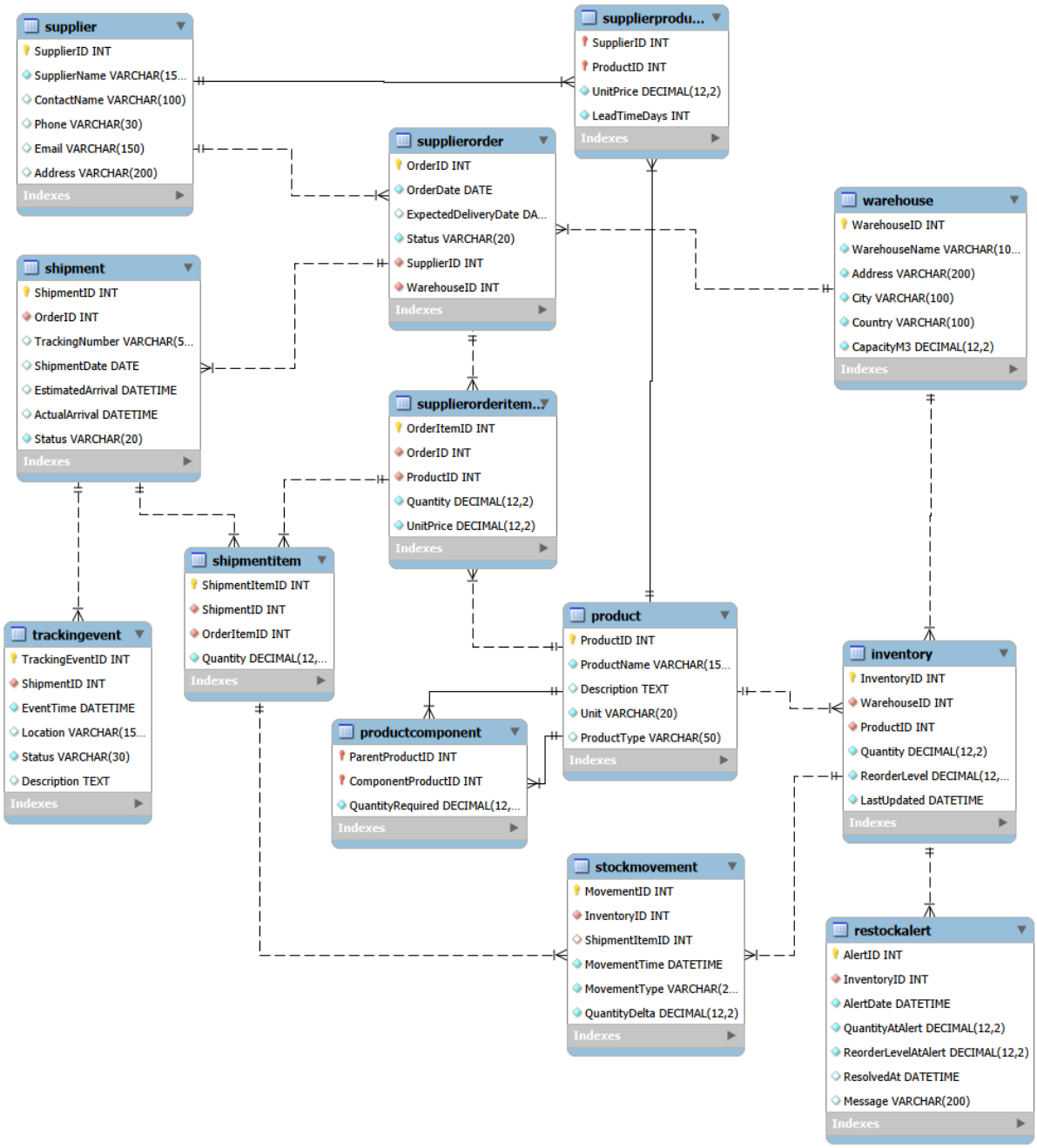
CONSTRAINT ck_restockalert CHECK (QuantityAtAlert < ReorderLevelAtAlert),

   CONSTRAINT fk_restockalert_inventory_id FOREIGN KEY (InventoryID) REFERENCES Inventory(InventoryID)

);

CREATE INDEX idx_alert_inventory_date ON RestockAlert(InventoryID, AlertDate) COMMENT 'Supports alert-history lookup for an inventory record';
```

II/ Entity-Relationship Diagrams (ERDs)



III/ Table Descriptions

Table 1. WAREHOUSE

1/ Purpose: Stores information about warehouses, including their names, locations, and storage capacities.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description

WarehouseID	INT	PK, NOT NULL, AUTO_INCREMENT	Unique identifier of a warehouse.
WarehouseName	VARCHAR(100)	NOT NULL	Name of the warehouse.
Address	VARCHAR(200)	NOT NULL	Physical address of the warehouse.
City	VARCHAR(100)	NOT NULL	City where the warehouse is located.
Country	VARCHAR(100)	NOT NULL	Country where the warehouse is located.
CapacityM3	DECIMAL(12,2)	NOT NULL, CHECK (CapacityM3>=0)	Maximum storage capacity in cubic meters. Must not be negative.

Table 2. PRODUCT

1/ Purpose: Stores information about products that can be purchased, stored, or assembled within the supply chain.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description
ProductID	INT	PK, NOT NULL	Unique identifier of a product.
ProductName	VARCHAR(150)	NOT NULL	Name used to identify the product.
Description	TEXT	NULL	Additional descriptive information about the product.
Unit	VARCHAR(20)	NOT NULL	Base unit used to measure the product quantity.
ProductType	VARCHAR(50)	NULL	Type or category of the product. Controlled list is finalized in Phase 2.

Table 3. INVENTORY

1/ Purpose: Records the current stock quantity and reorder level of each product at each warehouse.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description
InventoryID	INT	PK, NOT NULL	Unique identifier of an inventory record.
WarehouseID	INT	FK, NOT NULL	References Warehouse. Each inventory belongs to exactly one warehouse.

ProductID	INT	FK, NOT NULL	References Product. Each inventory belongs to exactly one product.
Quantity	DECIMAL(12,2)	NOT NULL, CHECK (Quantity>=0)	Current quantity available. Cannot be negative.
ReorderLevel	DECIMAL(12,2)	NOT NULL, CHECK (ReorderLevel>=0)	Minimum inventory threshold. Cannot be negative.
LastUpdated	DATETIME	NOT NULL	Date and time when the inventory balance was most recently changed.
Additional constraint : UNIQUE (WarehouseID, ProductID)			

Table 4. SUPPLIER

1/ Purpose: Stores information about suppliers that provide products to the organization.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description
SupplierID	INT	PK, NOT NULL	Unique identifier of a supplier.
SupplierName	VARCHAR(150)	NOT NULL	Name of the supplier.
ContactName	VARCHAR(100)	NULL	Name of the main contact person.
Phone	VARCHAR(30)	NULL	Supplier's contact telephone number.
Email	VARCHAR(150)	NULL	Supplier's contact email address.
Address	VARCHAR(200)	NULL	Supplier's business address.

Table 5. SUPPLIER_PRODUCT

1/ Purpose: Records the products offered by each supplier, including current unit prices and lead times.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description
SupplierID	INT	PK, FK, NOT NULL	References Supplier and forms part of the composite primary key.

ProductID	INT	PK, FK, NOT NULL	References Product and forms part of the composite primary key.
UnitPrice	DECIMAL(12,2)	NOT NULL, CHECK (UnitPrice >= 0)	Current price offered by the supplier.
LeadTimeDays	INT	NOT NULL, CHECK (LeadTimeDays >= 0)	Expected number of days required for delivery.

Table 6. SUPPLIER_ORDER

1/ Purpose: Stores purchase order information, including the supplier, destination warehouse, and order status.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description
OrderID	INT	PK, NOT NULL, AUTO_INCREMENT	Unique identifier of a supplier order.
OrderDate	DATE	NOT NULL	Date on which the supplier order is created.
ExpectedDeliveryDate	DATE	NULL	Expected delivery date at the destination warehouse.
Status	VARCHAR(20)	NOT NULL	Current lifecycle state of the supplier order.
SupplierID	INT	FK, NOT NULL	References Supplier. Each order refers to exactly one supplier.
WarehouseID	INT	FK, NOT NULL	References Warehouse. Each order has exactly one destination warehouse.

Table 7. SUPPLIER_ORDER_ITEM

1/ Purpose: Records individual product lines in a supplier order, including the ordered quantity and agreed unit price.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description

OrderItemID	INT	PK, NOT NULL, AUTO_INCREMENT	Unique identifier of an individual order line.
OrderID	INT	FK, NOT NULL	References SupplierOrder.
ProductID	INT	FK, NOT NULL	References Product.
Quantity	DECIMAL(12,2)	NOT NULL, CHECK (Quantity > 0)	Quantity of the product requested in the order line.
UnitPrice	DECIMAL(12,2)	NOT NULL, CHECK (UnitPrice >= 0)	Historical price agreed when the order was created.
Additional constraint : UNIQUE (OrderID, ProductID)			

Table 8. SHIPMENT

1/ Purpose: Stores information about shipments associated with supplier orders, including shipment dates, tracking numbers, and delivery status.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description
ShipmentID	INT	PK, NOT NULL, AUTO_INCREMENT	Unique identifier of a shipment.
OrderID	INT	FK, NOT NULL	References SupplierOrder. One order may have multiple shipments.
TrackingNumber	VARCHAR(50)	NULL	External tracking reference assigned to the shipment.
ShipmentDate	DATE	NULL	Date when the shipment is dispatched.
EstimatedArrival	DATETIME	NULL	Current expected arrival date and time.
ActualArrival	DATETIME	NULL	Confirmed arrival date and time. Empty until arrival is confirmed.
Status	VARCHAR(20)	NOT NULL	Current logistics state of the shipment.

Table 9. SHIPMENT_ITEM

1/ Purpose: Records the quantity of each supplier order item allocated to a shipment and supports partial deliveries.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description

ShipmentItemID	INT	PK, NOT NULL	Unique identifier of a shipment line.
ShipmentID	INT	FK, NOT NULL	References Shipment.
OrderItemID	INT	FK, NOT NULL	References SupplierOrderItem. Must belong to the same supplier order as the shipment.
Quantity	DECIMAL(12,2)	NOT NULL, CHECK (Quantity > 0)	Quantity allocated to this shipment. Must be positive. Total allocation cannot exceed the originally ordered quantity.

Table 10. TRACKING_EVENT

1/ Purpose: Records shipment tracking events, including their timestamps, locations, statuses, and descriptions.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description
TrackingEventID	INT	PK, NOT NULL, AUTO_INCREMENT	Unique identifier of a tracking event.
ShipmentID	INT	FK, NOT NULL	References Shipment.
EventTime	DATETIME	NOT NULL	Date and time when the tracking event occurs.
Location	VARCHAR(150)	NULL	Geographical location where the event is recorded.
Status	VARCHAR(30)	NOT NULL	Shipment status reported by the tracking event.
Description	TEXT	NULL	Additional information about the event or handling activity.

Table 11. PRODUCT_COMPONENT

1/ Purpose: Defines the component products and required quantities used to make or assemble parent products, supporting multi-level Bills of Materials (BOM).

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description

ParentProductID	INT	PK, FK, NOT NULL	References Product as the parent/assembled product.
ComponentProductID	INT	PK, FK, NOT NULL	References Product as the component product.
QuantityRequired	DECIMAL(12,2)	NOT NULL, CHECK (QuantityRequired > 0)	Number of component units required to produce one parent product. Must be greater than zero.
Additional constraint : CHECK (ParentProductID <> ComponentProductID)			

Table 12. STOCK_MOVEMENT

1/ Purpose: Records inventory transactions, such as receipts, issues, and adjustments, to maintain a history of stock changes.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description
MovementID	INT	PK, NOT NULL, AUTO_INCREMENT	Unique identifier of an inventory transaction.
InventoryID	INT	FK, NOT NULL	References Inventory.
ShipmentItemID	INT	FK, NULL, UNIQUE	Optional reference to ShipmentItem. Used only for Receipt movements.
MovementTime	DATETIME	NOT NULL	Date and time when the inventory transaction is recorded.
MovementType	VARCHAR(20)	NOT NULL	Category of inventory change, e.g. Opening, Receipt, Issue, Adjustment.
QuantityDelta	DECIMAL(12,2)	NOT NULL	Signed inventory change. Positive increases inventory; negative decreases inventory.

Table 13. RESTOCK_ALERT

1/ Purpose: Records low-stock alert periods when inventory falls below its reorder level and tracks when those alerts are resolved.

2/ Table structure:

Attribute	Data Type	Constraints / Notes	Description

AlertID	INT	PK, NOT NULL, AUTO_INCREMENT	Unique identifier of a low-stock alert period.
InventoryID	INT	FK, NOT NULL	References Inventory.
AlertDate	DATETIME	NOT NULL	Date and time when the low-stock condition is first detected.
QuantityAtAlert	DECIMAL(12,2)	NOT NULL	Inventory quantity captured when the alert is created. Historical snapshot.
ReorderLevelAtAlert	DECIMAL(12,2)	NOT NULL	Reorder level captured when the alert is created. Historical snapshot.
ResolvedAt	DATETIME	NULL	Date and time when the alert is resolved. NULL means unresolved.
Message	VARCHAR(200)	NULL	Human-readable explanation of the low-stock condition.

IV/ Index Documentation

1/ Index List

Index Name	Indexed Columns	Index Type	Purpose
idx_warehouse_name	WarehouseName	Non-unique	Supports searching warehouses by name.
idx_product_name	ProductName	Non-unique	Supports searching products by name.
idx_supplier_name	SupplierName	Non-unique	Supports searching suppliers by name.
idx_order_status_date	Status, OrderDate	Composite, non-unique	Supports filtering supplier orders

			by status and date.
idx_shipment_status	Status	Non-unique	Supports filtering shipments by status.
idx_tracking_shipment_time	ShipmentID, EventTime, TrackingEventID	Composite, non-unique	Supports retrieving tracking events in chronological order for a shipment.
idx_movement_inventory_time	InventoryID, MovementTime, MovementID	Composite, non-unique	Supports retrieving inventory movement history by inventory record and time.
idx_alert_inventory_date	InventoryID, AlertDate	Composite, non-unique	

V/ Constraints Documentation

1/ Primary Key Constraints

Table	Primary Key	Purpose
Warehouse	WarehouseID	Uniquely identifies each warehouse.
Product	ProductID	Uniquely identifies each product.
Supplier	SupplierID	Uniquely identifies each supplier.
Inventory	InventoryID	Uniquely identifies each inventory record.
SupplierProduct	SupplierID, ProductID	Identifies each supplier-product association.
SupplierOrder	OrderID	Uniquely identifies each supplier order.
SupplierOrderItem	OrderItemID	Uniquely identifies each supplier order line.
Shipment	ShipmentID	Uniquely identifies each shipment.

ShipmentItem	ShipmentItemID	Uniquely identifies each shipment line.
TrackingEvent	TrackingEventID	Uniquely identifies each tracking event.
ProductComponent	ParentProductID, ComponentProductID	Identifies each parent-product and component-product relationship.
StockMovement	MovementID	Uniquely identifies each stock movement.
RestockAlert	AlertID	Uniquely identifies each restock alert period.

2/ Foreign Key Constraints

Constraint Name	Table	Foreign Key Column	Referenced Table and Column
fk_inventory_warehouse_id	Inventory	WarehouseID	Warehouse(WarehouseID)
fk_inventory_product_id	Inventory	ProductID	Product(ProductID)
fk_supplierproduct_supplier_id	SupplierProduct	SupplierID	Supplier(SupplierID)
fk_supplierproduct_product_id	SupplierProduct	ProductID	Product(ProductID)
fk_order_supplier_id	SupplierOrder	SupplierID	Supplier(SupplierID)
fk_order_warehouse_id	SupplierOrder	WarehouseID	Warehouse(WarehouseID)
fk_orderitem_order_id	SupplierOrderItem	OrderID	SupplierOrder(OrderID)
fk_orderitem_product_id	SupplierOrderItem	ProductID	Product(ProductID)
fk_shipment_order_id	Shipment	OrderID	SupplierOrder(OrderID)
fk_shipmentitem_shipment_id	ShipmentItem	ShipmentID	Shipment(ShipmentID)
fk_shipmentitem_orderitem_id	ShipmentItem	OrderItemID	SupplierOrderItem(OrderItemID)
fk_trackingevent_shipment_id	TrackingEvent	ShipmentID	Shipment(ShipmentID)
fk_productcomponent_parent_id	ProductComponent	ParentProductID	Product(ProductID)
fk_productcomponent_component_id	ProductComponent	ComponentProductID	Product(ProductID)

fk_stockmovement_inventory_id	StockMovement	InventoryID	Inventory(InventoryID)
fk_stockmovement_shipmentitem_id	StockMovement	ShipmentItemID	ShipmentItem(ShipmentItemID)
fk_restockalert_inventory_id	RestockAlert	InventoryID	

3/ Unique Constraints

Constraint Name	Table	Columns	Purpose
uq_inventory_warehouseid_productid	Inventory	WarehouseID, ProductID	Ensures that each product has at most one inventory record in a given warehouse.
uq_orderitem_orderid_productid	SupplierOrderItem	OrderID, ProductID	Prevents the same product from appearing in multiple order lines within the same supplier order.
Unique constraint on ShipmentItemID	StockMovement	ShipmentItemID	Allows a shipment item to be associated with at most one stock movement through this column.

4/ Check Constraints

Table	Column(s)	Check Condition	Purpose
Warehouse	CapacityM3	CapacityM3 >= 0	Prevents negative warehouse capacity.
Inventory	Quantity	Quantity >= 0	Prevents negative inventory balances.
Inventory	ReorderLevel	ReorderLevel >= 0	Prevents negative reorder thresholds.
SupplierProduct	UnitPrice	UnitPrice >= 0	Prevents negative supplier prices.
SupplierProduct	LeadTimeDays	LeadTimeDays >= 0	Prevents negative delivery lead times.
SupplierOrderItem	Quantity	Quantity > 0	Ensures that the ordered quantity is positive.
SupplierOrderItem	UnitPrice	UnitPrice >= 0	Prevents negative order-line prices.
ShipmentItem	Quantity	Quantity > 0	Ensures that the allocated shipment quantity is positive.
ProductComponent	QuantityRequired	QuantityRequired > 0	Ensures that the

			required component quantity is positive.
ProductComponent	ParentProductID, ComponentProductID	ParentProductID <> ComponentProductID	Prevents a product from being its own direct component.
RestockAlert	QuantityAtAlert, ReorderLevelAtAlert	QuantityAtAlert < ReorderLevelAtAlert	Ensures that an alert represents a stock level below the recorded reorder threshold.

VI/ Version Control

Version control helps the team manage changes to the database design and SQL scripts. Whenever a change is made, the team should save the updated files and keep track of what has changed.

The team should keep the SQL scripts and database documentation organized so that members can easily find the latest version and avoid confusion when working together.

VII/ Regular Updates

The database documentation must be reviewed whenever a database object is added, modified, or removed.

The update process includes:

1/Updating the SQL scripts to reflect the schema changes.

2/Updating the ERD to represent the latest tables and relationships.

3/Revising table descriptions, indexes, and constraint documentation.

4/Testing the modified database objects to verify their correctness.

5/Recording the changes and the date of the latest revision.

This process helps ensure consistency between the database implementation and its documentation.

VIII/ Stored Procedures and Functions Documentation

Name	Type	Purpose
sp_RecordStockMovement	Procedure	Record a stock movement and update inventory
sp_ReceiveShipmentItem	Procedure	Receive a shipment item into the warehouse
sp_SubmitSupplierOrder	Procedure	Submit a draft supplier order
sp_AddShipmentItem	Procedure	Add an order line to a shipment
sp_SetInventoryReorderLevel	Procedure	Set the reorder level of an inventory record
fn_GetInventoryQuantity	Function	Return the current stock quantity
fn_IsInventoryLow	Function	Tell whether stock is below the reorder level

1/ Stored Procedure: sp_RecordStockMovement

Purpose: Records one stock movement (opening balance, receipt, issue or adjustment) and updates the matching inventory quantity.

Parameters:

-p_InventoryID (IN, INT): The inventory record whose quantity changes.

-p_MovementType (IN, VARCHAR(20)): Opening, Receipt, Issue or Adjustment.

-p_QuantityDelta (IN, DECIMAL(12,2)): Signed change; positive adds stock, negative removes it.

-p_ShipmentItemID (IN, INT): The shipment item being received; NULL for any movement that is not a receipt.

-p_MovementID (OUT, INT): ID of the new StockMovement row.

Description: The procedure checks the movement type, locks the Inventory row, and rejects the movement if the inventory record does not exist or the stock would become negative. Otherwise it inserts a row into StockMovement and adds p_QuantityDelta to Inventory.Quantity. Both writes run in one transaction.

SQL Code:

```
DELIMITER //
CREATE PROCEDURE sp_RecordStockMovement (
    IN  p_InventoryID    INT,
    IN  p_MovementType   VARCHAR(20),
    IN  p_QuantityDelta  DECIMAL(12,2),
    IN  p_ShipmentItemID INT,
    OUT p_MovementID     INT
)
BEGIN
    DECLARE v_Quantity DECIMAL(12,2);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
```

```
        ROLLBACK;
        RESIGNAL;
    END;
    IF p_MovementType NOT IN ('Opening', 'Receipt', 'Issue', 'Adjustment') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid movement type';
    END IF;
    START TRANSACTION;

    SELECT Quantity INTO v_Quantity
    FROM Inventory
    WHERE InventoryID = p_InventoryID
    FOR UPDATE;
    IF v_Quantity IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Inventory record not
found';
    END IF;
    IF v_Quantity + p_QuantityDelta < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient stock';
    END IF;
    INSERT INTO StockMovement (InventoryID, ShipmentItemID, MovementType,
QuantityDelta)
    VALUES (p_InventoryID, p_ShipmentItemID, p_MovementType,
p_QuantityDelta);
    SET p_MovementID = LAST_INSERT_ID();
    UPDATE Inventory
    SET Quantity = Quantity + p_QuantityDelta
    WHERE InventoryID = p_InventoryID;
    COMMIT;
END //
DELIMITER ;
```

2/ Stored Procedure: sp_ReceiveShipmentItem

Purpose: Receives a shipment item at its destination warehouse and increases the stock by the shipped quantity.

Parameters:

-p_ShipmentItemID (IN, INT): The shipment line being received.

-p_MovementID (OUT, INT): ID of the Receipt movement that was created.

Description: The procedure checks that the shipment item exists and has not been received yet (each shipment item can have only one Receipt movement). It finds the destination warehouse through Shipment and SupplierOrder, and the product through SupplierOrderItem. It creates the Inventory row if the warehouse has none for that product, then calls sp_RecordStockMovement with type Receipt and the shipped quantity.

SQL Code:

```
DELIMITER //
CREATE PROCEDURE sp_ReceiveShipmentItem (
   IN  p_ShipmentItemID INT,
   OUT p_MovementID     INT
)
BEGIN
   DECLARE v_Quantity    DECIMAL(12,2);
   DECLARE v_WarehouseID INT;
   DECLARE v_ProductID   INT;
   DECLARE v_InventoryID INT;

   SELECT si.Quantity, o.WarehouseID, oi.ProductID
   INTO v_Quantity, v_WarehouseID, v_ProductID
   FROM ShipmentItem si
   JOIN Shipment s           ON s.ShipmentID   = si.ShipmentID
   JOIN SupplierOrder o      ON o.OrderID      = s.OrderID
   JOIN SupplierOrderItem oi ON oi.OrderItemID = si.OrderItemID
   WHERE si.ShipmentItemID = p_ShipmentItemID;
   IF v_Quantity IS NULL THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Shipment item not
found';
   END IF;
   IF EXISTS (SELECT 1 FROM StockMovement WHERE ShipmentItemID =
p_ShipmentItemID) THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Shipment item already
received';
   END IF;
   INSERT IGNORE INTO Inventory (WarehouseID, ProductID)
   VALUES (v_WarehouseID, v_ProductID);
   SELECT InventoryID INTO v_InventoryID
   FROM Inventory
   WHERE WarehouseID = v_WarehouseID AND ProductID = v_ProductID;
   CALL sp_RecordStockMovement(v_InventoryID, 'Receipt', v_Quantity,
p_ShipmentItemID, p_MovementID);
END //
DELIMITER ;
```

3/ Stored Procedure: sp_SubmitSupplierOrder

Purpose: Submits a supplier order by changing its status from Draft to Submitted.
Parameters:
-p_OrderID (IN, INT): The supplier order to submit.
-Output: none.

Description: The procedure checks that the order exists, is still in Draft status, and has at least one line in SupplierOrderItem. If all checks pass, it sets SupplierOrder.Status to Submitted.

SQL Code:

```
DELIMITER //
CREATE PROCEDURE sp_SubmitSupplierOrder (
    IN p_OrderID INT
)
BEGIN
    DECLARE v_Status VARCHAR(20);
    SELECT Status INTO v_Status
    FROM SupplierOrder
    WHERE OrderID = p_OrderID;
    IF v_Status IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Order not found';
    END IF;
    IF v_Status <> 'Draft' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only Draft orders can
be submitted';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM SupplierOrderItem WHERE OrderID =
p_OrderID) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Order has no items';
    END IF;
    UPDATE SupplierOrder
    SET Status = 'Submitted'
    WHERE OrderID = p_OrderID;
END //
DELIMITER ;
```

4/ Stored Procedure: sp_AddShipmentItem

Purpose: Adds a line of a supplier order to a shipment.

Parameters:

-p_ShipmentID (IN, INT): The shipment to add the item to.
-p_OrderItemID (IN, INT): The supplier order line being shipped.
-p_Quantity (IN, DECIMAL(12,2)): Quantity to put in this shipment.
-p_ShipmentItemID (OUT, INT): ID of the new ShipmentItem row.

Description: The procedure checks that the shipment is in Draft status and that the order line belongs to the same supplier order as the shipment. It then checks that the quantity already placed in shipments plus the new quantity does not exceed the ordered quantity, and inserts the ShipmentItem row.

SQL Code:

```
DELIMITER //
CREATE PROCEDURE sp_AddShipmentItem (
    IN  p_ShipmentID     INT,
    IN  p_OrderItemID    INT,
    IN  p_Quantity       DECIMAL(12,2),
    OUT p_ShipmentItemID INT
)
BEGIN
    DECLARE v_Status     VARCHAR(20);
    DECLARE v_OrderedQty DECIMAL(12,2);
    DECLARE v_ShippedQty DECIMAL(12,2);

    SELECT s.Status, oi.Quantity
    INTO v_Status, v_OrderedQty
    FROM Shipment s
    JOIN SupplierOrderItem oi ON oi.OrderID = s.OrderID
    WHERE s.ShipmentID = p_ShipmentID
      AND oi.OrderItemID = p_OrderItemID;
    IF v_Status IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Shipment or order item not found, or they belong to
different orders';
    END IF;
    IF v_Status <> 'Draft' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only Draft shipments
can be changed';
    END IF;
    SELECT COALESCE(SUM(Quantity), 0) INTO v_ShippedQty
    FROM ShipmentItem
    WHERE OrderItemID = p_OrderItemID;
    IF v_ShippedQty + p_Quantity > v_OrderedQty THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Quantity exceeds the
ordered quantity';
    END IF;
    INSERT INTO ShipmentItem (ShipmentID, OrderItemID, Quantity)
    VALUES (p_ShipmentID, p_OrderItemID, p_Quantity);

    SET p_ShipmentItemID = LAST_INSERT_ID();
END //
DELIMITER ;
```

5/ Stored Procedure: sp_SetInventoryReorderLevel

Purpose: Sets the stock level below which a product at a warehouse needs to be reordered.
Parameters:
-p_InventoryID (IN, INT): The inventory record to update.
-p_NewReorderLevel (IN, DECIMAL(12,2)): The new reorder level; must not be negative.
-Output: none.
Description: The procedure rejects a negative level or an unknown inventory record, then updates Inventory.ReorderLevel.
SQL Code:

```
DELIMITER //
CREATE PROCEDURE sp_SetInventoryReorderLevel (
    IN p_InventoryID     INT,
    IN p_NewReorderLevel DECIMAL(12,2)
)
BEGIN
    IF p_NewReorderLevel < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Reorder level must not
be negative';
    END IF;

    IF NOT EXISTS (SELECT 1 FROM Inventory WHERE InventoryID =
p_InventoryID) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Inventory record not
found';
    END IF;

    UPDATE Inventory
    SET ReorderLevel = p_NewReorderLevel
    WHERE InventoryID = p_InventoryID;
END //
DELIMITER ;
```

6/ Function: fn_GetInventoryQuantity

Purpose: Returns the current stock quantity of an inventory record.
Parameters:
-p_InventoryID (IN, INT): The inventory record to look up.
Return value: DECIMAL(12,2), the value of Inventory.Quantity; NULL if the record does not exist.
Description: The function reads Quantity from Inventory for the given InventoryID and returns it.
SQL Code:

```
CREATE FUNCTION fn_GetInventoryQuantity (p_InventoryID INT)
RETURNS DECIMAL(12,2)
READS SQL DATA
RETURN (SELECT Quantity FROM Inventory WHERE InventoryID =
p_InventoryID);
```

7/ Function: fn_IsInventoryLow

Purpose: Tells whether the stock of an inventory record is below its reorder level.

Parameters:

- p_InventoryID (IN, INT): The inventory record to check.
- Return value: TINYINT(1); 1 if Quantity < ReorderLevel, 0 otherwise, NULL if the record does not exist.

Description: The function compares Quantity with ReorderLevel of the given inventory record. The same rule decides when a RestockAlert is opened.

SQL Code:

```
CREATE FUNCTION fn_IsInventoryLow (p_InventoryID INT)
RETURNS TINYINT(1)
READS SQL DATA
RETURN (SELECT Quantity < ReorderLevel FROM Inventory WHERE InventoryID
= p_InventoryID);
```